

Proposta de Trabalho Final: Rede Social

"I want to believe"



Desenvolvimento de Aplicativos Móveis com Android **Tecnologias**→ Kotlin, Jetpack Compose, Firebase (Authentication, Cloud Firestore, Cloud Storage)

1. Visão Geral do Projeto

O objetivo deste trabalho final é aplicar de forma prática os conceitos de desenvolvimento Android moderno na construção de uma aplicação completa e funcional. Os alunos deverão desenvolver uma rede social simplificada, que chamaremos de **"I want to believe"**.

A aplicação permitirá que usuários criem uma conta, personalizem seus perfis, publiquem postagens com texto e imagem, e visualizem um feed com as publicações de outros usuários em tempo real. O uso do Firebase como backend (BaaS - Backend as a Service) garantirá a persistência dos dados, autenticação segura e o armazenamento de mídias na nuvem, simulando um ambiente de produção real.

2. Objetivos de Aprendizagem

Ao concluir este projeto, o aluno deverá ser capaz de:

- Estruturar um projeto Android moderno utilizando arquitetura recomendada (ex: MVVM).
- Construir interfaces de usuário (UI) reativas e declarativas com **Jetpack Compose**.
- Implementar um fluxo completo de autenticação de usuários com **Firebase Authentication**.
- Modelar, persistir e consultar dados em um banco de dados NoSQL com **Cloud Firestore**.
- Realizar o upload e download de arquivos de mídia (imagens) utilizando **Firebase Cloud Storage**.
- Gerenciar estados da UI e operações assíncronas de forma eficiente, utilizando **StateFlow/SharedFlow** e Coroutines.
- Implementar a navegação entre telas de forma coesa com o **Jetpack Navigation Compose**.

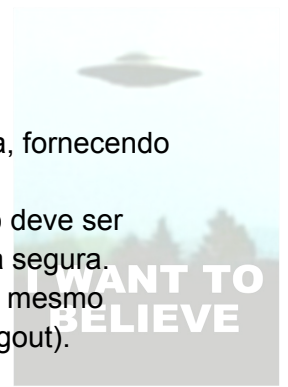
3. Requisitos Mínimos da Aplicação

A aplicação deve, obrigatoriamente, conter as seguintes funcionalidades:

a) Autenticação de Usuário

- **Tela de Login:** Campos para e-mail e senha para que um usuário existente possa acessar sua conta.

- **Tela de Cadastro:** Formulário para que um novo usuário crie uma conta, fornecendo nome, e-mail e senha.
- **Integração com Firebase Authentication:** A lógica de login e cadastro deve ser gerenciada pelo Firebase. As senhas devem ser armazenadas de forma segura.
- **Gerenciamento de Sessão:** O aplicativo deve manter o usuário logado mesmo após ser fechado e reaberto. Deve haver uma opção clara de "Sair" (Logout).



b) Perfil do Usuário

- **Criação de Perfil no Firestore:** No momento do cadastro, um documento representando o perfil do usuário deve ser criado na coleção **users** do Cloud Firestore. Este documento deve conter, no mínimo, o **uid** (do Firebase Auth), o nome e o e-mail do usuário.
- **Tela de Perfil:** Uma tela que exibe as informações do usuário logado (nome e e-mail).

c) Feed de Publicações

- **Tela Principal (Feed):** Deve exibir uma lista de todas as publicações feitas pelos usuários, em ordem cronológica (da mais nova para a mais antiga).
- **Atualização em Tempo Real (opcional: *Clicando em um comando*):** O feed deve ser atualizado automaticamente quando novas publicações forem adicionadas no Firestore.
- **Layout da Publicação:** Cada item no feed deve exibir claramente o nome do autor, a imagem da postagem e o texto da descrição.

d) Criação de Publicações

- **Tela de Nova Publicação:** Uma tela com um campo de texto para a descrição e um botão para selecionar uma imagem da galeria do dispositivo.
- **Upload de Imagem para o Cloud Storage:** A imagem selecionada pelo usuário deve ser enviada para um bucket no Firebase Cloud Storage.
- **Persistência da Publicação no Firestore:** Após o sucesso do upload da imagem, um novo documento deve ser criado em uma coleção **posts** no Firestore. Este documento deve conter:
 - A URL da imagem armazenada no Storage.
 - O texto da descrição.
 - O **uid** do autor da postagem.
 - Um **timestamp** com a data e hora da criação.

4. Requisitos Técnicos

- **Linguagem:** O projeto deve ser desenvolvido 100% em **Kotlin**.
- **Interface de Usuário:** A UI deve ser construída inteiramente com **Jetpack Compose**. O uso de XML layouts não é permitido.
- **Arquitetura:** É obrigatório o uso do padrão **MVVM** (Model-View-ViewModel) para organizar a lógica da aplicação.

- **Navegação:** A navegação entre as telas deve ser implementada com o **Jetpack Navigation Compose**.

5. Funcionalidades Adicionais (Pontos Extras, Será negociado)

Para os alunos que desejarem aprofundar seus conhecimentos e obter uma pontuação extra, sugere-se a implementação de uma ou mais das seguintes funcionalidades:

- **Sistema de "Likes":** Permitir que usuários curtam as publicações. A contagem de likes deve ser exibida e atualizada em tempo real.
- **Seção de Comentários:** Adicionar uma funcionalidade para que os usuários possam ver e adicionar comentários nas publicações.
- **Edição de Perfil:** Permitir que o usuário atualize seu nome e adicione uma foto de perfil (que também deve ser salva no Cloud Storage).
- **Exclusão de Publicações:** Permitir que o autor de uma publicação possa excluí-la do feed.
- **Tratamento de Erros e Carregamento:** Exibir indicadores de carregamento (loading spinners) enquanto os dados são buscados e tratar erros de conexão ou falhas de forma elegante, mostrando mensagens amigáveis ao usuário.

6. Critérios de Avaliação

Os projetos serão avaliados com base na correta implementação de cada um dos requisitos mínimos:

- **Autenticação de Usuário (25%):** Implementação completa do fluxo de login, cadastro e gerenciamento de sessão.
- **Perfil do Usuário (25%):** Criação do perfil no Firestore no momento do cadastro e exibição correta dos dados.
- **Feed de Publicações (25%):** Exibição da lista de publicações em tempo real.
- **Criação de Publicações (25%):** Funcionalidade de upload de imagem e persistência da publicação no banco de dados.

7. Instruções de Entrega

- Apresentação em sala de aulas dos aplicativos em data a ser definida em conjunto com a turma

Data de Entrega: 04 e 10 de Setembro de 2025

